

▼ STEP 1:IMPORTING RELEVANT LIBRARIES

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import io
from google.colab import files
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import adjusted_rand_score, mutual_info_score
import seaborn as sns
```

▼ STEP 2: EXPLORATORY DATA ANALYSIS

```
uploaded=files.upload()
```

[Choose Files](#) streaming c...features.csv

streaming customers features.csv(text/csv) - 11869 bytes, last modified: 5/1/2026 - 100% done
Saving streaming customers features.csv to streaming customers features.csv

```
seg= pd.read_csv(io.BytesIO(uploaded['streaming customers features.csv']))
```

```
seg.head()
```

	customer_id	monthly_hours_watched	monthly_spend_usd	content_diversity_score	
0	C0000	44.03	21.23	2.13	
1	C0001	9.92	10.78	3.34	
2	C0002	34.83	21.09	1.94	
3	C0003	82.63	17.80	5.26	
4	C0004	70.44	33.99	6.92	

Next steps: [Generate code with seg](#) [New interactive sheet](#)

```
seg.describe()
```

	monthly_hours_watched	monthly_spend_usd	content_diversity_score	
count	500.000000	500.000000	500.000000	
mean	53.716800	20.671840	3.963480	
std	32.127642	9.785165	2.642481	
min	4.680000	5.000000	0.000000	
25%	23.640000	13.202500	1.980000	
50%	50.615000	18.805000	3.045000	
75%	80.117500	25.912500	6.235000	
max	122.930000	44.410000	10.000000	

500 Data points with 3 features - (list them) For each feature, list the min and max value and interpret them

```
seg.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 4 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customer_id           500 non-null   object
1   monthly_hours_watched 500 non-null   float64
```

```

2  monthly_spend_usd      500 non-null    float64
3  content_diversity_score 500 non-null    float64
dtypes: float64(3), object(1)
memory usage: 15.8+ KB

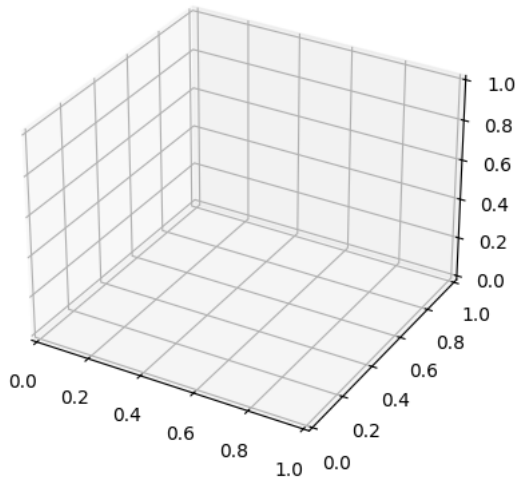
```

Visualization of the data in a 3-d scatter plot

```

fig = plt.figure()
ax = fig.add_subplot(projection='3d')

```



```

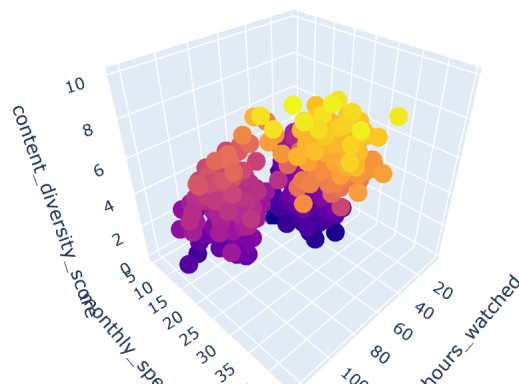
import plotly.express as px
from IPython.display import HTML

```

```

fig = px.scatter_3d(seg, x='monthly_hours_watched', y='monthly_spend_usd', z='content_diversity_score', color='content_diversity_score')
HTML(fig.to_html())

```



```

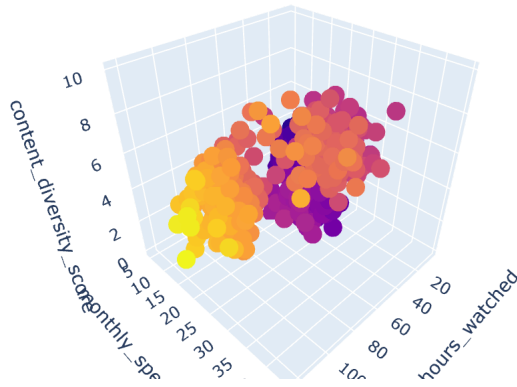
import plotly.express as px
from IPython.display import HTML

```

```

fig = px.scatter_3d(seg, x='monthly_hours_watched', y='monthly_spend_usd', z='content_diversity_score', color='monthly_hours_watched')
HTML(fig.to_html())

```



✓ #3. Using the Elbow method to identify best number of clusters

Creating an array to fit the K means and Drop customer ID because its an object

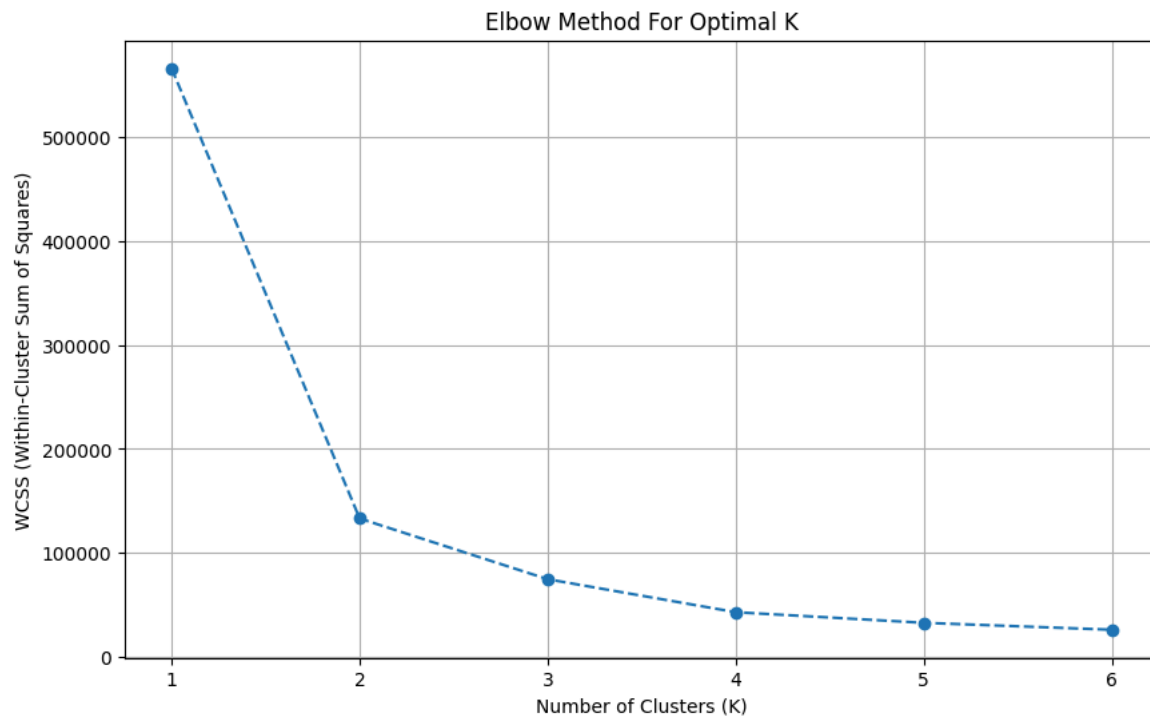
```
X= seg.drop('customer_id', axis=1)
```

```
X=np.array(X)
```

```
wcss = [] # Within-cluster sum of squares
for i in range(1, 7):
    kmeans = KMeans(n_clusters=i, init='k-means++', max_iter=300, n_init=10, random_state=0)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)

# Plotting the elbow method graph
plt.figure(figsize=(10, 6))
plt.plot(range(1, 7), wcss, marker='o', linestyle='--')
plt.title('Elbow Method For Optimal K')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('WCSS (Within-Cluster Sum of Squares)')
plt.grid(True)
plt.show()

print("Observe the 'elbow point' in the graph to determine the optimal K.")
```



Observe the 'elbow point' in the graph to determine the optimal K.

we chose a K of 3 as the inertia score is changing the most

Start coding or [generate](#) with AI.

4. IMPLEMENTING K MEANS CLUSTERING ON K=3 CLUSTERS

```
kmeans = KMeans(n_clusters=3, init='k-means++', max_iter=300, n_init=10, random_state=0)
```

```
labels= kmeans.fit_predict(X)
```

```
seg['clusterlabel']=labels
```

```
seg.head()
```

	customer_id	monthly_hours_watched	monthly_spend_usd	content_diversity_score	clusterlabel	
0	C0000	44.03	21.23	2.13	0	
1	C0001	9.92	10.78	3.34	0	
2	C0002	34.83	21.09	1.94	0	
3	C0003	82.63	17.80	5.26	2	
4	C0004	70.44	33.99	6.92	1	

Next steps:

[Generate code with seg](#)

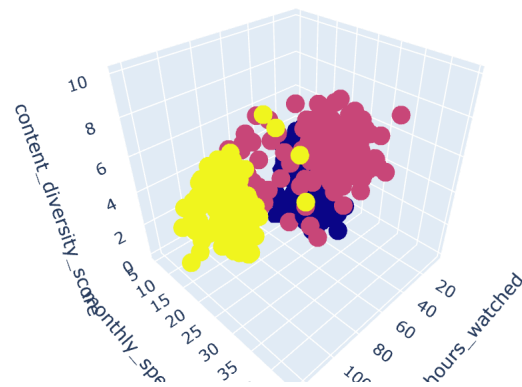
[New interactive sheet](#)

finding centroids for each cluster

```
kmeans.cluster_centers_
```

```
array([[24.60149798, 14.86935223, 2.03076923],
       [70.58288591, 31.7852349 , 7.11228188],
       [98.70173077, 18.53067308, 4.04240385]])
```

```
fig = px.scatter_3d(seg, x='monthly_hours_watched', y='monthly_spend_usd', z='content_diversity_score', color='clusterlabel')
HTML(fig.to_html())
```



```
centroids = kmeans.cluster_centers_
centroids_df = pd.DataFrame(centroids, columns=['monthly_hours_watched', 'monthly_spend_usd', 'content_diversity_score'])
centroids_df['clusterlabel'] = range(len(centroids_df))

fig = px.scatter_3d(seg, x='monthly_hours_watched', y='monthly_spend_usd', z='content_diversity_score', color='clusterlabel', opacity=0.5)

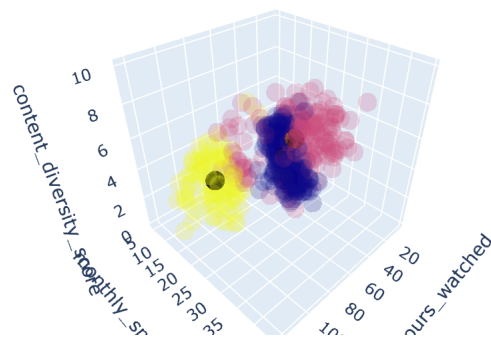
# Create the centroid trace without the 'marker' argument that caused the error
centroid_trace = px.scatter_3d(centroids_df, x='monthly_hours_watched', y='monthly_spend_usd', z='content_diversity_score',
                               color='clusterlabel', symbol=np.full(len(centroids_df), 'diamond'), symbol_sequence=['circle']).data

# Now, set the marker properties directly on the trace object
centroid_trace.marker.size = 8
centroid_trace.marker.color = 'black'

centroid_trace.marker.sizemode = 'diameter'
centroid_trace.marker.line = dict(width=2, color='black')

fig.add_trace(centroid_trace)
HTML(fig.to_html())
```

Customer Segments with Centroids



5. Cluster Analysis

```
group0=seg[seg['clusterlabel']==0]
```

```
group1=seg[seg['clusterlabel']==1]
group2=seg[seg['clusterlabel']==2]
```

```
group0.describe()
```

	monthly_hours_watched	monthly_spend_usd	content_diversity_score	clusterlabel	
count	247.000000	247.000000	247.000000	247.0	
mean	24.601498	14.869352	2.030769	0.0	
std	11.418552	5.691299	1.046877	0.0	
min	4.680000	5.000000	0.000000	0.0	
25%	14.320000	9.880000	1.315000	0.0	
50%	23.250000	14.000000	2.080000	0.0	
75%	34.270000	19.750000	2.720000	0.0	
max	49.670000	26.930000	7.170000	0.0	

```
group1.describe()
```

	monthly_hours_watched	monthly_spend_usd	content_diversity_score	clusterlabel	
count	149.000000	149.000000	149.000000	149.0	
mean	70.582886	31.785235	7.112282	1.0	
std	9.244885	8.366662	1.982668	0.0	
min	48.200000	10.130000	0.520000	1.0	
25%	64.420000	29.140000	6.640000	1.0	
50%	70.630000	34.560000	7.670000	1.0	
75%	77.560000	37.220000	8.380000	1.0	
max	90.490000	44.410000	10.000000	1.0	

```
group2.describe()
```

	monthly_hours_watched	monthly_spend_usd	content_diversity_score	clusterlabel	
count	104.000000	104.000000	104.000000	104.0	
mean	98.701731	18.530673	4.042404	2.0	
std	8.873773	4.679054	1.502007	0.0	
min	80.070000	9.570000	1.100000	2.0	
25%	92.407500	15.562500	2.995000	2.0	
50%	98.475000	17.735000	4.050000	2.0	
75%	103.277500	20.615000	4.842500	2.0	
max	122.930000	39.260000	9.470000	2.0	

```
group0['monthly_spend_usd'].sum()
```

```
np.float64(3672.73)
```

```
group1['monthly_spend_usd'].sum()
```

```
np.float64(4736.0)
```

```
group2['monthly_spend_usd'].sum()
```

```
np.float64(1927.19)
```

Visualizing the clusters! -boxplots for monthly hours watched, boxplots for monthly spend usd, boxplots for content diversity score (all 3 clusters on one graph **per feature**) use seaborn for it Pie chart for total income split between the clusters

Engineering and Plotting a New Feature: Spend per Diversity Score

✓ Combined Boxplots for Cluster Features (All in One Figure)

```
fig, axes = plt.subplots(1, 3, figsize=(25, 7), sharey=False)

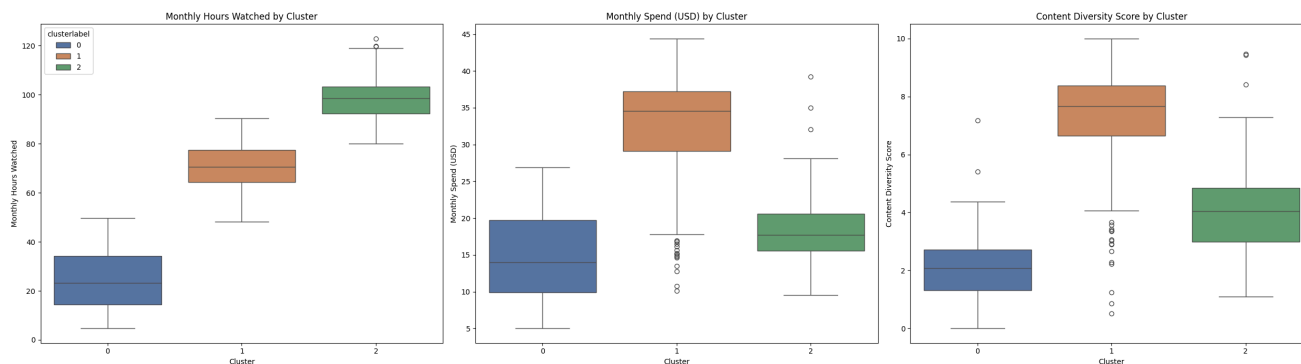
# Define a consistent palette for the clusters
# Seaborn will automatically map the colors to cluster labels 0, 1, 2
consistent_cluster_palette = sns.color_palette('deep', n_colors=3)

# Monthly Hours Watched
sns.boxplot(x='clusterlabel', y='monthly_hours_watched', data=seg, palette=consistent_cluster_palette, ax=axes[0], hue='cluster')
axes[0].set_title('Monthly Hours Watched by Cluster')
axes[0].set_xlabel('Cluster')
axes[0].set_ylabel('Monthly Hours Watched')
# Seaborn automatically adds a legend when `hue` is used. We'll only keep one.

# Monthly Spend (USD)
sns.boxplot(x='clusterlabel', y='monthly_spend_usd', data=seg, palette=consistent_cluster_palette, ax=axes[1], hue='clusterlabel')
axes[1].set_title('Monthly Spend (USD) by Cluster')
axes[1].set_xlabel('Cluster')
axes[1].set_ylabel('Monthly Spend (USD)')

# Content Diversity Score
sns.boxplot(x='clusterlabel', y='content_diversity_score', data=seg, palette=consistent_cluster_palette, ax=axes[2], hue='cluster')
axes[2].set_title('Content Diversity Score by Cluster')
axes[2].set_xlabel('Cluster')
axes[2].set_ylabel('Content Diversity Score')

plt.tight_layout()
plt.show()
```

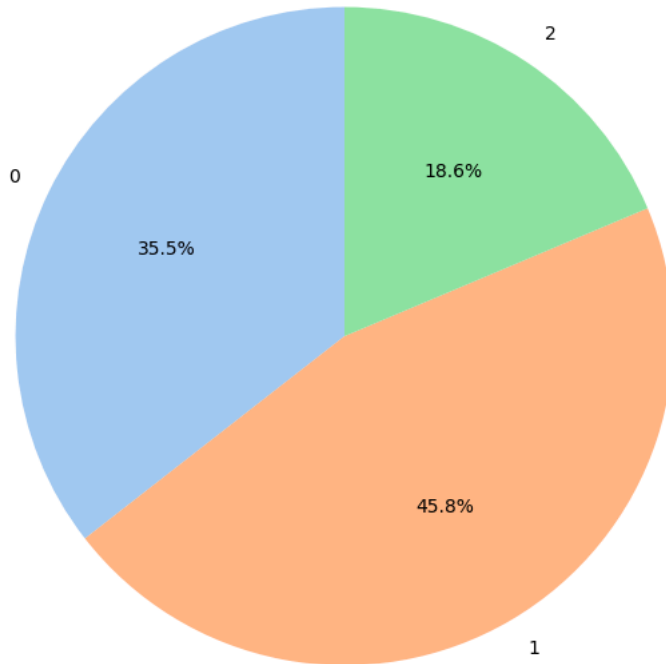


✓ Pie Chart for Total Income Split

```
total_spend_by_cluster = seg.groupby('clusterlabel')['monthly_spend_usd'].sum()

plt.figure(figsize=(8, 8))
plt.pie(total_spend_by_cluster, labels=total_spend_by_cluster.index, autopct='%1.1f%%', startangle=90, colors=sns.color_palette('deep', n_colors=3))
plt.title('Total Monthly Spend (USD) Split by Cluster')
plt.show()
```

Total Monthly Spend (USD) Split by Cluster



Plotting each feature against eachother to see cluster correlations

monthly spend per diverity for each cluster

Start coding or [generate](#) with AI.

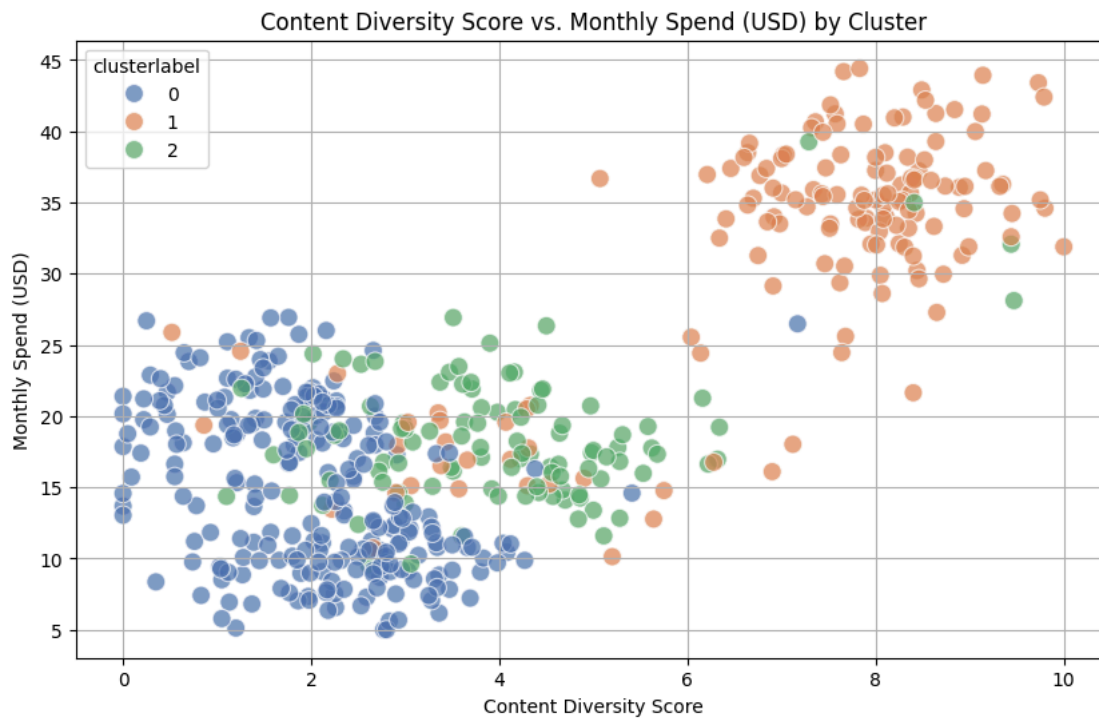
The `spend_per_diversity` feature represents how much a customer spends relative to the diversity of content they watch. A higher value suggests they spend more for each unit of content diversity.

These scatter plots help visualize:

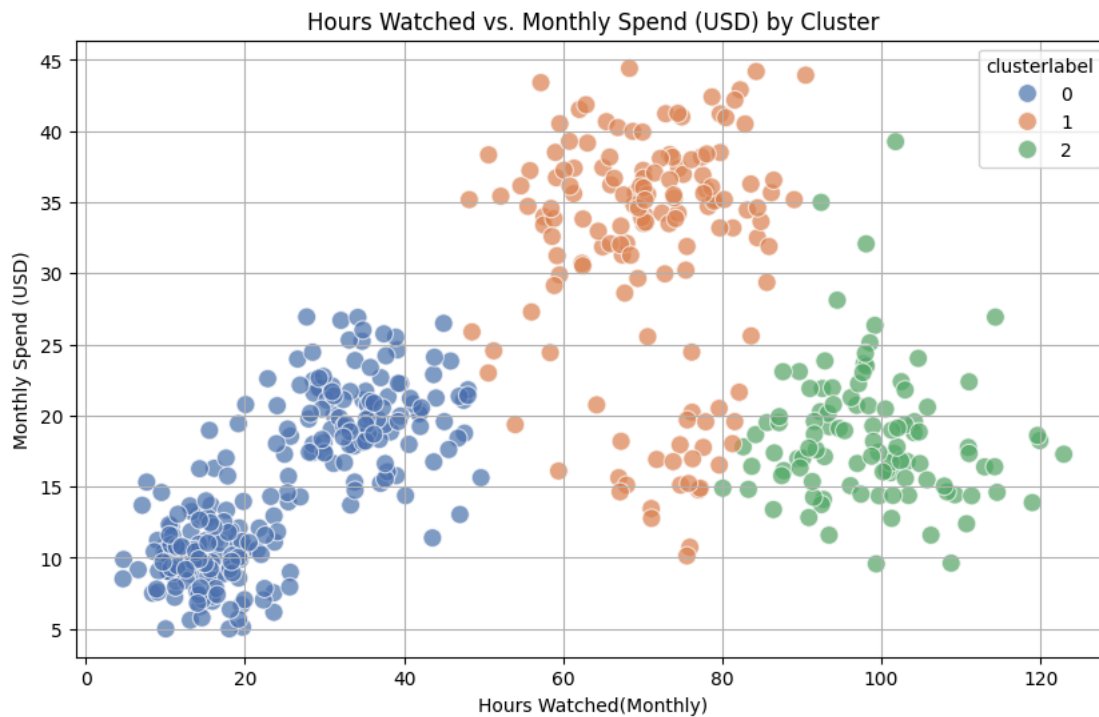
- The relationship between a customer's `content_diversity_score` and their `spend_per_diversity`.
- The relationship between a customer's `monthly_spend_usd` and their `spend_per_diversity`.

By coloring the points by `clusterlabel`, we can observe if the clusters show distinct patterns or correlations in these new relationships, potentially revealing different customer behaviors regarding spending and content exploration. Any `NaN` values resulting from division by zero in `content_diversity_score` are automatically excluded from these plots.

```
plt.figure(figsize=(10, 6))
sns.scatterplot(x='content_diversity_score', y='monthly_spend_usd', data=seg, hue='clusterlabel', palette='deep', s=100, alpha=0.5)
plt.title('Content Diversity Score vs. Monthly Spend (USD) by Cluster')
plt.xlabel('Content Diversity Score')
plt.ylabel('Monthly Spend (USD)')
plt.grid(True)
plt.show()
```



```
plt.figure(figsize=(10, 6))
sns.scatterplot(x='monthly_hours_watched', y='monthly_spend_usd', data=seg, hue='clusterlabel', palette='deep', s=100, alpha=0.7)
plt.title('Hours Watched vs. Monthly Spend (USD) by Cluster')
plt.xlabel('Hours Watched(Monthly)')
plt.ylabel('Monthly Spend (USD)')
plt.grid(True)
plt.show()
```



```
# Create the new feature: monthly_spend_usd per content_diversity_score
# Handle potential division by zero if content_diversity_score can be 0 by replacing with NaN
seg['spend_per_diversity'] = seg['monthly_spend_usd'] / seg['content_diversity_score'].replace(0, np.nan)

# Display the head of the DataFrame with the new feature
display(seg.head())
```

	customer_id	monthly_hours_watched	monthly_spend_usd	content_diversity_score	clusterlabel	spend_per_diversity
0	C0000	44.03	21.23	2.13	0	9.967136
1	C0001	9.92	10.78	3.34	0	3.227545
2	C0002	34.83	21.09	1.94	0	10.871134
3	C0003	82.63	17.80	5.26	2	3.384030
4	C0004	70.44	33.99	6.92	1	4.911850

This boxplot visualizes the distribution of `spend_per_diversity` for each cluster. The 'difference' can be observed by comparing the medians (the line inside the box), the interquartile ranges (the boxes themselves), and the spread (whiskers) across the three clusters. This helps to identify which clusters have higher or lower `spend_per_diversity` and how consistent that behavior is within each cluster.

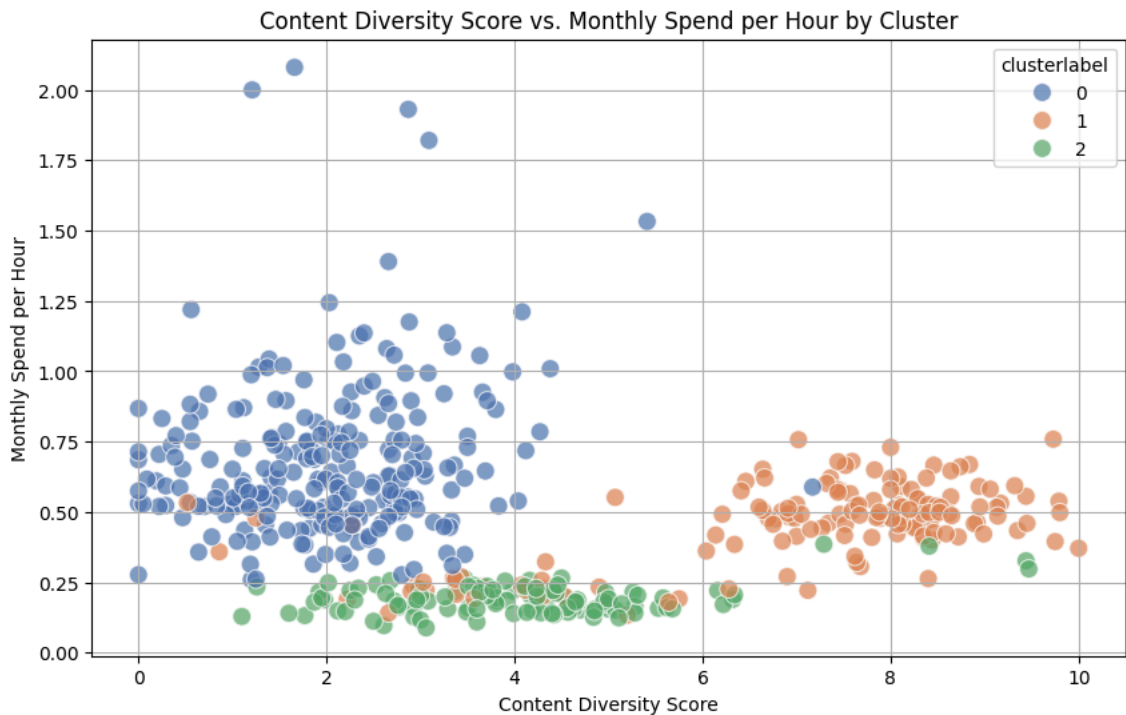
Double-click (or enter) to edit

```
seg['monthly_spend_per_hour'] = seg['monthly_spend_usd'] / seg['monthly_hours_watched']
seg.head()
```

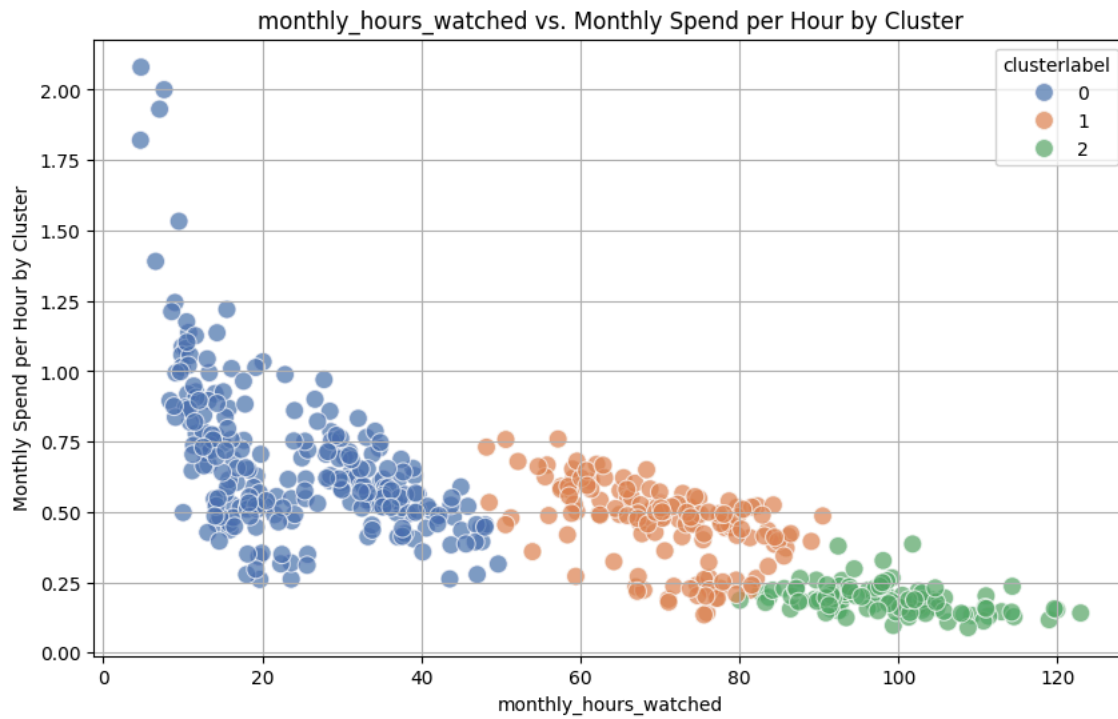
	customer_id	monthly_hours_watched	monthly_spend_usd	content_diversity_score	clusterlabel	spend_per_diversity	monthly_sp
0	C0000	44.03	21.23	2.13	0	9.967136	
1	C0001	9.92	10.78	3.34	0	3.227545	
2	C0002	34.83	21.09	1.94	0	10.871134	
3	C0003	82.63	17.80	5.26	2	3.384030	
4	C0004	70.44	33.99	6.92	1	4.911850	

Next steps: [Generate code with seg](#) [New interactive sheet](#)

```
plt.figure(figsize=(10, 6))
sns.scatterplot(x='content_diversity_score', y='monthly_spend_per_hour', data=seg, hue='clusterlabel', palette='deep', s=100, a
plt.title('Content Diversity Score vs. Monthly Spend per Hour by Cluster')
plt.xlabel('Content Diversity Score')
plt.ylabel('Monthly Spend per Hour')
plt.grid(True)
plt.show()
```



```
plt.figure(figsize=(10, 6))
sns.scatterplot(x='monthly_hours_watched', y='monthly_spend_per_hour', data=seg, hue='clusterlabel', palette='deep', s=100, alpha=0.5)
plt.title('monthly_hours_watched vs. Monthly Spend per Hour by Cluster')
plt.xlabel('monthly_hours_watched')
plt.ylabel('Monthly Spend per Hour by Cluster')
plt.grid(True)
plt.show()
```



Cluster analysis- results

```
group0.describe()
```

	monthly_hours_watched	monthly_spend_usd	content_diversity_score	clusterlabel	
count	247.000000	247.000000	247.000000	247.0	
mean	24.601498	14.869352	2.030769	0.0	
std	11.418552	5.691299	1.046877	0.0	
min	4.680000	5.000000	0.000000	0.0	
25%	14.320000	9.880000	1.315000	0.0	
50%	23.250000	14.000000	2.080000	0.0	
75%	34.270000	19.750000	2.720000	0.0	
max	49.670000	26.930000	7.170000	0.0	

```
group1.describe()
```

monthly_hours_watched monthly_spend_usd content_diversity_score clusterlabel

group2.describe()

	monthly_hours_watched	monthly_spend_usd	content_diversity_score	clusterlabel
mean	70.582886	31.785235	7.112282	1.0
count	104.000000	104.000000	104.000000	104.0
min	48.200000	10.130000	0.520000	1.0
max	98.701731	18.530673	4.042404	2.0
std	8.873773	4.679054	1.502007	0.0
50%	70.630000	34.560000	7.670000	1.0
25%	92.407500	15.562500	2.995000	2.0
max	90.490000	44.410000	10.000000	1.0
50%	98.475000	17.735000	4.050000	2.0
75%	103.277500	20.615000	4.842500	2.0
max	122.930000	39.260000	9.470000	2.0

Characteristics of each group: 0-Low amount of hours watched, least money spent, least diverse 1- big spenders, long watchers, very diverse 2-super long watchers, low money spent, low diversity.

smallwatchers =group0
smallwatchers.describe()

	monthly_hours_watched	monthly_spend_usd	content_diversity_score	clusterlabel
count	247.000000	247.000000	247.000000	247.0
mean	24.601498	14.869352	2.030769	0.0
std	11.418552	5.691299	1.046877	0.0
min	4.680000	5.000000	0.000000	0.0
25%	14.320000	9.880000	1.315000	0.0
50%	23.250000	14.000000	2.080000	0.0
75%	34.270000	19.750000	2.720000	0.0
max	49.670000	26.930000	7.170000	0.0

bigwatchersandspenders=group1
bigwatchersandspenders.describe()

	monthly_hours_watched	monthly_spend_usd	content_diversity_score	clusterlabel
count	149.000000	149.000000	149.000000	149.0
mean	70.582886	31.785235	7.112282	1.0
std	9.244885	8.366662	1.982668	0.0
min	48.200000	10.130000	0.520000	1.0
25%	64.420000	29.140000	6.640000	1.0
50%	70.630000	34.560000	7.670000	1.0
75%	77.560000	37.220000	8.380000	1.0
max	90.490000	44.410000	10.000000	1.0

longbingers=group2
longbingers.describe()

	monthly_hours_watched	monthly_spend_usd	content_diversity_score	clusterlabel
count	104.000000	104.000000	104.000000	104.0